→ Characteristics of an algorithm:—

Any algorithm must satisfy the following properties:

1. Input: It generally requires finite no of inputs.
2. Output: It must produce at least one output.
3. Finiteness: It must terminate after a finite no of steps
4. Definiteness: Each instruction should be clear & unambiguous
5. Effectiveness: Clear & effective analysis of steps should be sufficiently basic.

→ Analysis of algorithm:—

- It refers to the task of determining how much computing time & storage an algorithm require.
- The efficiency of computer algorithm can be measured in terms of 2 qualities:
  1. Time complexity
  2. Space complexity

Space complexity:—

- Amount of memory usage space a program takes to run to its completion.
- Space needed by each algo is the sum of following components
  - Fixed Part (fixed space requirement):
    It include instruction space (space for storing program)
    space for simple variables, constants etc.
  - Variable part (variable space requirement): It includes space for variable whose size vary with changes in input & space for referenced variables + recursion stack space
- c is a constant representing the fixed space requirement and $S_p(I)$ is the variable space requirement of a program P working on an instant I.
- Total space requirement $S(P) = C + S_p(I)$

E.g:

```
* Sum (a,n)                    i → 1 space
  {                            n → 1 space
   for (i=0 to n)              A[i] → n space
     {
      A[i] = 0                 ∴ S(P) = 1+1+n = 2+n
     }                                  O(n)
  }
```

```
* abc (a,b,c)                  a → 1
  {                            b → 1
   return a+b*c+(a+b-c)/(a+b)*0  c → 1
  }                                  3 space
```

```
* for (i=0; i<n; i++)
    for (j=0; j<n; j++)
      {
       c[i][j] = a[i][j] + b[i][j];
      }
```

i → 1 j → 1 n → 1 c[i][j] → n×n b[i][j] → n×n a[i][j] → n×n

$S(P) = 1+1+1 + n\times n + n\times n + n\times n = 3 + 3n^2 = 3(n^2+1)$

$O(n^2)$

Time Complexity :—

- It measures how much time an algo requires to run the program completely.
- Total time T(P) taken by a program P is the sum of its compile & run time.
- Time complexity only consider execution time.
- Time complexity count the no of instruction steps performed by the algorithm.
- Generally, time complexity of an algorithm expressed using big O notation. The time complexity is normally estimated by counting the number of elementary operations performed by the algorithm, then express the efficiency of algorithm using growth function.

Growth fn / Estimate no of steps in algo as it's input grows.

Growth fn	Elementary Operation
O(1)	Time requirement is constant if it is independent of the problem's size.
O(n)	Time requirement for a linear algorithm increases directly with the size of the pblm.
$O(n^2)$	Time requirement for a quadratic algo increases rapidly with the size of the pblm. (n^3 → cubic)
$O(2^n)$	size of problem increases, the time requirement for an exponential algo
O(n log n)	Time requirement increases rapidly than a linear algo.
$O(\log_2 n)$	Time requirement for a logarithmic algo increases slowly as the pblm size increases.

- Frequency count: how many times certain instruction is executed in a piece of code (the no. of operations/steps).
- To find time complexity of a given program — summing the frequency counts of all statements of that program.
- Every simple statement in an algorithm takes one unit of time.

* Rules:

- Comment and declaration has frequency count = 0.
- Return & assignment stmnt has frequency count = 1.
- Ignore low order exponent when higher order exponents are present.
  - ignore constant multipliers.
  e.g. $5n^4 + 7n^3 + 10n^2 + n + 100$
  $O(n^7)$
- For looping stmnts — no. of times loop is repeating.

Example :—

```
Sum()                          0
{                              0
 // this is an example         0
 a = 5                         1
 b = 10                        1
 c = a + b                     1
}                              0
                               3 ⇒ O(1)
```

"If there is no iteration/recursion in the algo then the complexity is order of 1 i.e. constant"

↑ Lowest

- If a program does not contain iteration or recursion, there is no need to find out time complexity for the same.
- It will always be a constant as no of parameters keep on changing.

Example

① abc(a, b, c) (0)
{ (0)
return a+b+c+(a+b+c)/(a+b)+4.0(1) (0)
} (0)

TC = O(1)

② A(a, b)
{
temp = a 1
a = b 1
b = temp 1
} 3 → constant → O(1)

③ Sum(A, n)
{
s = 0 1
for(i=0; i<n; i++) n+1
{
s = s + A[i]; n
}
return s 1
}

2n+3 TC = O(n)

		n=5		
	i=0	0<5	true	1st
	i=1	1<5	*	*
6	i=2	2<5	*	*
n+1	i=3	3<5	*	*
	i=4	4<5	*	*
	i=5	5<5		

④ add(a, n)
{
for(i=0; i<n; i++) → n+1
{
for(j=0; j<n; j++) → n(n+1)
{
s[i][j] = a[i][j] + b[i][j] n×n = n²
}
}
}

n+1 = n+1
n(n+1) = n²+n
n×n = n²

2n²+2n+1 ⇒ O(n²)

⑤ fb(n)
{
if n≤1 1
print n 1
else
{
a=0 1
b=1 1
for(i=2; i≤n; i++) n
{
c=a+b n-1
a=b n-1
b=c n-1
}
Print c 1
}
}

n=5

i=2	2<5	i++	n-1
i=3	3<5	i++	
i=4	4<5	i++	
i=5	5<5	i++	
i=6			

n
5

4n+1 O(n)

⑥
```
matmul()
{
  for(i=0; i<n; i++)        ----> n+1      = n+1
  {
    for(j=0; j<n; j++)      ----> n(n+1)   = n²+n
    {
      c[i][j]=0             ----> n×n      = n²
      for(k=0; k<n; k++)    ----> n n (n+1) = n³+n²
      {
        c[i][j] = a[i][k]*c[i][j]+b[k][j]   n n n = n³
      }
    }
  }
```

$2n^3+3n^2+2n+1$

$TC = O(n^3)$

⑦
```
Sum(a[], n, m)
{
  for i=0 to n      ----> n+1
    for j=0 to m    ----> n(m+1)
      s = s+a[i][j] ----> nm
  return s          ----> 1
}
```

$2nm+2n+2$

* Computational model: Used to analysing different algo by placing algo. it to model with some no. of inputs & it can be analysed how much of time taken for giving proper output.

* RAM (Random access machine model): 1st computational model.

→ Asymptotic Notation: —

- shorthand way to write down the fastest possible and slowest possible running times of algorithm.
- i.e, best case, worst case, and average case of an algorithm expressed using asymptotic notations.

→ Different cases of algorithm complexity: —

① Best case: Ω
- The function defined in minimum number of steps taken by any input of size n.
- Minimum time taken by an algorithm to run to its completion
- Denoted by Ω

② Average case: Θ
- The function defined in average number of steps taken by any input of size n.
- Average time taken by an algorithm to run to its completion
- Denoted by Θ

③ Worst case: O
- The function defined by the ~~algorithm is the~~ function of maximum number of steps taken by any input of size n.
- maximum time for completion
- Denoted by O.

Search(a, n, x)	Best	worst	Average
{			
for i=1 to n	1	n+1	n/2
if a[i] = x	1	n	n/2
return i	1	0	1
return -1	0	1	0
}	3	2n+1	n+1
	Ω(1)	O(n)	Θ(n)

dominant term: term that grows the fastest

* Best case: Searched element is the 1st element in that list
* Average case: Search data in the middle of list
* Worst case: Visit every element but element is not present in that list.

→ Asymptotic Notations: —

- Mathematical notations used to describe the running time of an algorithm (mathematical way to represent time complexity)
- Notations are defined in terms of functions.

* Types: —

- Big-Oh Notation (O)
- Big-Omega Notation (Ω)
- Big Theta Notation (Θ)
- Little Oh Notation (o)
- Little Omega Notation (ω)

- For large i/p size it is sufficient to analyse the behaviour of the dominant term. An analysis is conducted for large values of i/p size is called asymptotic analysis.
- Asymptotic notation short hand way to express the best case, average case, worst case complexity of an algorithm.

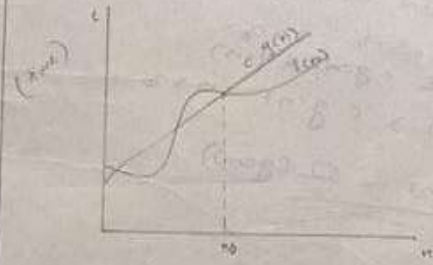
1. Big-Oh Notation: —

- For non negative functions f(n) and g(n), if there exists an integer n_0 and a constant $c > 0$ such that for all integers $n > n_0$

$$0 \le f(n) \le c g(n) \quad \forall\ n \ge n_0$$

→ $f(n) = O(g(n))$

→ $f(n) \le c \cdot g(n)$ $(c > 0,\ n_0 \ge 1)$.



f(n) = O(g(n))

g(n) is an asymptotic upper bound for f(n)

- We use it to find what is the worst case or upper bound of the function f(n)

- The x axis of the above graph ... the input (n) and y axis showing the increase in time (t).

- If we bound the time complexity or f(n) of a given algorithm in terms of input n with another function called c g(n) for all the input size greater than n_0, then we can say that the f(n) is upper bound with the fn g(n) with the multiple of some constant c.

- f(n) = O(g(n)) → f(n) is smaller than c g(n)

2. Big Omega Notation: (Ω):—

- Used to express the lower bound of an algorithm.

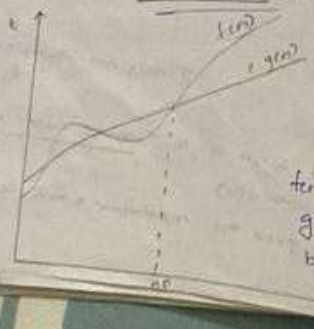
- i.e, best case of that algorithm.

- Definition:— For non-negative fn f(n) & g(n), if there exist an integer n_0 & a constant c>0 such that, ∀ $n \geq n_0$.

$0 \leq c\,g(n) \leq f(n)$

$f(n) \geq c\,g(n)$ $n \geq n_0$

$\Rightarrow f(n) = \Omega(g(n))$



f(n) = Ω g(n)

g(n) is an asymptotic lower bound for f(n).

- The graph shows the time complexity analysis of Ω notation, it indicates the minimum time bound required for an algorithm for all input values.

3. Big Theta notation Θ:—

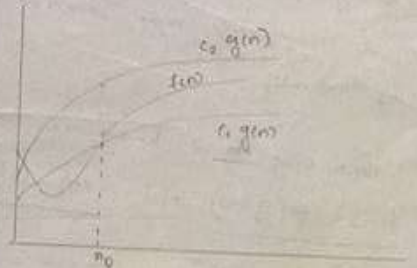
- The lower and upper bound for the function is provided by the theta notation.

- Definition: For non-negative functions f(n) & g(n), if there exist an integer n_0 & positive constants c_1 and c_2 [i.e $c_1, c_2 > 0$] such that for all integers $n \geq n_0$

$c_1 g(n) \leq f(n) \leq c_2 g(n)$

$\Rightarrow f(n) = \Theta(g(n))$

- If f(n) = Θ(g(n)) → f(n) = O(g(n)) since theta gives tighter bound.



4. Little oh Notation:—

- Tightless upper bound.

- For non-negative functions f(n) & g(n), if there exist an integer n_0 & constant c>0 such that for all integer $n \geq n_0$

$0 \leq f(n) \leq c\,g(n)$ ∀ $n \geq n_0$.

$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

$g(n)$ becomes arbitrarily large relative to $f(n)$ as n approaches infinity.

- It is asymptotically loose upper bound.

5 Little omega:- (ω)

- It is tightless lower bound / asymptotically loose lower bound.
- The function $f(n) = \omega(g(n))$ iff for any positive integer n_0 and a constant $c > 0$, such that

$f(n) \geq c\, g(n) \geq 0 \quad \forall n \geq n_0$

$\Rightarrow f(n) = \omega(g(n))$

$\Rightarrow \lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$

$f(n)$ become arbitrarily large relative to $g(n)$ as n approaches infinity.

Theorem: for any two functions $f(n)$ & $g(n)$ we have $f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ & $f(n) = \Omega(g(n))$.

→ Properties of Asymptotic Notation:—

* Reflexivity:
  - $f(n) = O(f(n))$
  - $f(n) = \Theta(f(n))$
  - $f(n) = \Omega(f(n))$
* Symmetry: (only for average case)
  - $f(n) = \Theta(g(n))$ iff $g(n) = \Theta(f(n))$
* Transpose symmetry: (best & worst case)
  - $f(n) = O(g(n))$ iff $g(n) = \Omega(f(n))$
  - $f(n) = o(g(n))$ iff $g(n) = \omega(f(n))$
* Transitivity:
  - $f(n) = O(g(n))$ and $g(n) = O(h(n)) \rightarrow f(n) = O(h(n))$
  - $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n)) \Rightarrow f(n) = \Theta(h(n))$
  - $f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n)) \Rightarrow f(n) = \Omega(h(n))$
  - $f(n) = o(g(n))$ and $g(n) = o(h(n)) \Rightarrow f(n) = o(h(n))$
  - $f(n) = \omega(g(n))$ and $f(n) = \omega(h(n)) \Rightarrow f(n) = \omega(h(n))$

→ Common complexity functions:—

* Constant time
  An algo is said to be constant time if the value of $f(n)$ is bounded by a value that doesn't depend on size of input
  - Computation time is constant.
  - E.g. $O(1)$.
* Logarithmic time:
  - An algo is said to be logarithmic time if $O(\log n)$.

* Linear time:
  If $f(n) = O(n)$

* Quadratic time
  $f(n) = O(n^2)$

* polynomial time
  $f(n) = O(n^k)$

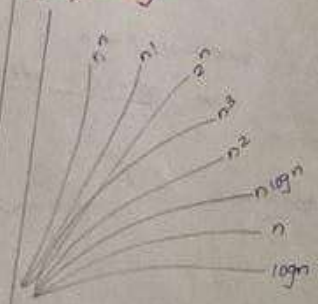
* Exponential time
  $f(n) = O(2^n)$

* Factorial time
  $f(n) = O(n!)$

Notation	order of growth
$O(1)$	Constant
$O(\log n)$	Logarithmic
$O(n)$	Linear
$O(n \log n)$	Log linear
$O(n^2)$	Quadratic
$O(n^3)$	Cubic
$O(2^n)$ $O(10^n)$	Exponential.

$O(1) < O(\log n) < O(n) < O(n^2) < O(2^n) < O(n!)$.

- The worst case(O) analysis is the important analysis among all.
- While the average case (Θ) only performed when a problem has similar upper & lower bound time complexity.



$O(1) < O(\log n) < O(n) < O(n \log n)$
$< O(n^2) < O(n^3) < O(2^n) < O(n!) < O(n^n)$

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n! < 2^n < n^n$

→ Applications! —

- O notation is worst case complexity most the algo never exceed this complexity with any large amount of input
- Ω notation is best case complexity. with any small input or program never execute less than this time
- Θ notation is average case complexity

→ Example:—

1. Find big oh notation for following functions.

(a) $f(n) = 5n^3 + n^2 + 6n + 2$

⇒ for big oh notation: $f(n) \leq c \cdot g(n)$.

$5n^3 + n^2 + 6n + 2 \leq c \cdot g(n)$

If $n \geq 2$ then 2 replace by n

$5n^3 + n^2 + 6n + 2 \leq 5n^3 + n^2 + 6n + n$
$\leq 5n^3 + n^2 + 7n$

If $n^2 \geq 7n$

$5n^3 + n^2 + 6n + 2 \leq 5n^3 + n^2 + n^2$
$\leq 5n^3 + 2n^2$.

If $n^3 \geq 2n^2$

$5n^3 + n^2 + 6n + 2 \leq 5n^3 + n^3$
$\leq 6n^3$ ⇒ $c \cdot g(n)$
$= n^3$

If $f(n) \leq c \cdot g(n)$

$f(n) = O(g(n)) = O(n^3)$

(b) $4n^3+2n+3 = f(n)$ $\quad n=1$ $\quad 1+2+3 \le 5 \quad 9 \le n$

$4n^3+2n+3 \le c\cdot g(n)$ $\quad n=2$ $\quad 39 \le 40$ T

$n \ge 3$ $\quad n=3$

$4n^3+2n+3 \le 4n^3+3n \quad 117 \le 135$ T

$n^3 \ge 3n$

$4n^3+2n+3 \le 5n^3 \Rightarrow c\cdot g(n) \quad n_0 = 2$

$f(n) = O(n^3)$ $\quad n \ge 2$

Find out the lower bound for $f(n) = 27n^2+16n+25$

⇒ lower bound ⇒ big omega ⇒ $f(n) \ge c\cdot g(n)$ ⇒ th

$g(n) = n^2 \quad c = 27$

$27n^2+16n+25 \ge 27n^2 \Rightarrow c\cdot g(n)$ $\quad n \ge n_0$

$n=1$ $\quad 68$ $\quad 27$ T

2 $\quad 165$ $\quad 108$ T

$n \ge 1$ $\quad f(n) = \Omega(n^2)$

Find the tighter bound for $f(n) = 27n^2+16n$

⇒ tighter bound ⇒ Θ notation ⇒ $c_1 g(n) \le f(n) \le c_2 g(n)$

Here we want to find both Ω, O notation

$\underbrace{c_1 g(n) \le f(n)}_{\Omega} \le \underbrace{c_2 g(n)}_{O}$

Big Oh notation $\quad 27 \quad 28n^2$

$f(n) \le c_1 g(n) \quad c_1 g(n) = 28n^2$

$27n^2+16n \le c_1 g(n)$

$n^2 \ge 16n$

$27n^2+16n \le 27n^2+n^2$

$\le 28n^2 \Rightarrow c_1 g(n)$

$f(n) = O(n^2)$ ⟶ ①

Big omega notation

$c_1 g(n) \le f(n) \Rightarrow f(n) \ge c_1 g(n)$

$27n^2+16n \ge c_1 g(n)$

$27n^2+16n \ge 27n^2 \Rightarrow c_1 g(n)$

$f(n) = \Omega(n^2)$ ⟶ ②

From ① & ②

$f(n) = \Theta(n^2)$

Is $2^{n+1} = O(2^n)$ $\quad f(n) \le c\cdot g(n)$

$f(n) \quad g(n)$

⇒ Big oh notation ⇒ $f(n) = c\cdot g(n)$.

We want to prove:

$2^{n+1} \le c\cdot(2^n)$

take some values to substitute.

$n = 0 \quad c = 1$

$2^1 = 1 \times 2^0$

$2 \neq 1$

$n = 1 \quad c = 1$

$2^2 = 1 \times 2^1$

$4 \neq 2$

then $n = 0 \quad c = 2$

$2^1 = 2 \times 2^0$

$2 = 2$

$n = 1, \quad c = 2$

$2^2 = 2 \times 2^1$

$4 = 4$

then it will be same for $c = 2$

$\therefore \quad c \geq 2 \quad n \geq 0$

$\therefore \quad 2^{n+1} = O(2^n)$

4. is $2^{2n} = O(2^n)$

$\Rightarrow f(n) \leq O(g(n)) \leq c \cdot g(n)$

$f(n) = 2^{2n} \quad g(n) = 2^n$

We Want to prove

$2^{2n} \leq c \cdot 2^n$

$2^{2n} = 2^n 2^n$

$= 2^{n+n}$

$= 2^{2n}$

$2^n \cdot 2^n \leq c \cdot 2^n$

$2^n \leq c$

When n increases c also varies $\therefore$ We can't prove this because we can't set a value for c is difficult

ie, $2^{2n} = O(2^n)$ is false

6. prove $n^2 + n = O(n^3)$

$g(n) = n^3 \quad f(n) = n^2 + n$

$f(n) \leq c \cdot g(n)$

$n^2 + n \leq c \cdot n^3$

$n = 1 \quad c = 1$

$1^2 + 1 \leq 1 \cdot 1^3 \Rightarrow 2 \leq 1$ false

$n = 1 \quad c = 2$

$1^2 + 1 \leq 2 \cdot 1^3 \Rightarrow 2 \leq 2$ true

$n = 2 \quad c = 2$

$2^2 + 1 \leq 2 \cdot 2^3 \Rightarrow 5 \leq 16$ true

$\therefore \quad n^2 + n = O(n^3) \quad n \geq 1 \quad c \geq 1$

7. Find tight tt bound for $f(n) = 3n^3 + 4n^2 + 5n + 6$

→ Find O & Ω notation

$\Theta \rightarrow \underbrace{c_1 g(n) \le f(n)}_{\Omega} \le \underbrace{c_2 g(n)}_{O}$

Big Oh

$f(n) \le c_2 g(n)$

$3n^3 + 4n^2 + 5n + 6 \le c_2 g(n)$

$n \ge 6$

$3n^3 + 4n^2 + 5n + 6 \le 3n^3 + 4n^2 + 6n$

$n^2 \ge 6n$

$3n^3 + 4n^2 + 5n + 6 \le 3n^3 + 5n^3$

$n^3 \ge 5n^2$

$3n^3 + 4n^2 + 5n + 6 \le 3n^3 + n^3$

$\le 4 \cdot n^3$

$c = 4 \quad g(n) = n^3$

$f(n) = O(n^3) \rightarrow (1)$

Big omega $f(n) \ge c_1 g(n)$

$3 \cdot n^3 + 4n^2 + 5n + 0 \ge 3n^3$

$c = 3 \quad g(n) = n^3$

$f(n) = \Omega(n^3) \rightarrow (2)$

From (1) & (2)

$f(n) = \Theta(n^3)$

8. Θ notation for $3n + 2$

→ $\Theta \rightarrow c_1 g(n) \le f(n) \le c_2 g(n)$

$f(n) \le c_2 g(n)$

$3n + 2 \le c_2 g(n)$

$3n \ge 2$

$3n + 2 \le 4n \rightarrow c_2 g(n)$

$f(n) =$

$3n \le 3n + 2 \rightarrow c_1 g(n) \le f(n)$

$c_1 g(n) \le f(n) \le c_2 g(n) \rightarrow f(n) = \Theta(g(n))$

$3n \le 3n + 2 \le 4n \Rightarrow f(n) = \Theta(n)$

$c_1 = 3 \quad c_2 = 4 \quad g(n) = n$

P.T $n! = \Theta(n^n)$ ✓

$f(n) = n!$

$g(n) = n^n$

→ $\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0 \qquad 0 \qquad \lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$

$\lim_{n \to \infty} \frac{n!}{n^n} = 0$ ✓

$n! = n(n-1)(n-2) \cdots 2 \cdot 1$

$= n^n [1 \cdot (1-1/n)(1-2/n) \cdots 2/n \cdot 1/n]$

$= n^n [(1-1/n)(1-2/n) \cdots 2/n \cdot 1/n]$

$\lim_{n \to \infty} \frac{n!}{n^n} = \lim_{n \to \infty} \frac{n^n ((1-1/n)(2/n) \cdots 2/n \cdot 1/n)}{n^n}$

$= \lim_{n \to \infty} (1-1/n)(1-2/n) \cdots 2/n \cdot 1/n) = (1-1/\infty)(1-2/\infty) \cdots 2/\infty \cdot 1/\infty)$

$= (0-0)(1-0) \cdots 0 \cdot 0) = 0$

→ Recurrence

→ Recursive Algorithm! —

→ Recurrence!

A recurrence is an equation that describes a function in terms of its value on smaller inputs.

E.g. $T(n) = \begin{cases} \Theta(n) & \text{if } n=1 \\ 2T(n/2) + \Theta(n) & \text{if } n>1 \end{cases}$

Time me 4 method for solving recurrence

- ✓ Substitution method
- ✓ Iteration method
- ✓ Recursion tree method
- ✓ Master's theorem

Example

```
RSum(a,n)
{
 if (n<=0)
   return 0
 else
   return a[n] + RSum(a,n-1)
}
```

$$T(n) = \begin{cases} 2 & n<=0 \\ 2+T(n-1) & n>0 \end{cases}$$

```
Fib(n)
{
 if (n==0||n==1)
   return n;
 else
   return (fib(n-1) + fib(n-2));
}
```

$$T(n) = \begin{cases} 1 & n=0 \text{ or } n=1 \\ 3 + T(n-1) + T(n-2) & n>1 \end{cases}$$

① Iteration method! —

- Base idea is to expand the recurrence and express it as summation of terms dependent only on n and initial condition
- T(1) will be the ground condition

① $T(n) = \begin{cases} 2 & n=0 \\ T(1) & n=1 \\ 2 + T(n-1) & n>0 \end{cases}$

$T(n) = 2 + T(n-1) \rightarrow ①$

$T(n-1) = 2 + T((n-1)-1) = 2 + T(n-2) \rightarrow ②$

② in ①

$T(n) = 2 + 2 + T(n-2) = 4 + T(n-2)$

$T(n-2) = 2 + T(n-2-1) = 2 + T(n-3)$

$T(n) = 4 + 2 + T(n-3) = 6 + T(n-3) = 2+2+2+T(n-3)$

$= 2k + [T(n-k)]$

If $k = n$

$T(n) = 2n + T(n-n)$ $\quad T(0) = 2$

$= 2n + T(0)$

$= 2n + 2$

$= O(n)$

2. $T(n) = \begin{cases} 1 & n = 1 \\ n + 2T(n/2) & n > 1 \end{cases}$

$\Rightarrow T(n) = n + 2T(n/2)$

$T(n/2) = n/2 + 2T(n/4)$

$T(n) = n + (2(n/2 + 2T(n/4)))$

$= n + (n + 4T(n/4)) = 2n + 4T(n/4)$

$T(n/4) = n/4 + 2T(n/8)$

$T(n) = 2n + 4(n/4 + 2T(n/8))$

$= 2n + n + 8T(n/8) = 3n + 8T(n/8)$

$= 3n + 2^3 T(n/2^3)$

$T(n) = kn + 2^k T(n/2^k)$

$T(1) = 1$

$n/2^k = 1$

$n = 2^k$ take log $\quad \log n = k$

$\log n = \log 2^k \rightarrow k \cdot \log 2$

$T(n) = n \log n + 2^{\log n} T(\log n)$

$= n \log n + 2^{\log_2 n}$ $\quad a^{\log_c b} = b^{\log_c a}$

$= n \log n + n \log_2^2 = n \log n + n$

$O(n \log n)$

$\log_b n = k \cdot \log_2 2$

$\log n = k \cdot 1$

$\log n = k$

3. $T(n) = 3T(n/4) + n$

$\Rightarrow T(n) = 3T(n/4) + n$

$T(n/4) = 3T(n/16) + n/4$

$T(n) = 3 \cdot 3T(n/16) + 3n/4 + n$

$= 9T(n/16) + \frac{3n}{4} + n$

$T(n/16) = 3T(n/64) + n/16$

$T(n) = 9\left[3T(n/64) + n/16\right] + \frac{3n}{4} + n$

$= 27T(n/64) + 9n/16 + \frac{3n}{4} + n$

$= 3^3 T(n/4^3) + \frac{3^2 n}{4^2} + \frac{3^1 n}{4^1} + \frac{3^0 n}{4^0}$

$T(n) = 3^k T(n/4^k) + n\left[1 + 3/4 + (3/4)^2 + \cdots\right]$

$T(1) = 1$

$n/4^k = 1$

$n = 4^k$

$\log n = \log 4^k$

$\log_4^n = k \log 4$

$\log_4^n = k$

$\log a^b = b \log a$

$\log^a \log b = \log_b^a$

$a + ar + ar^2 + \cdots = \frac{a}{1-r}$

$T(n) = 3^{\log_4^n} T(1) + n\left(1 + (3/4) + (3/4)^2 + \cdots\right)$

$= 3^{\log_4^n} T(1) + n\left(\frac{1}{1-3/4}\right)$ $\quad 1/1-3/4$

$= n^{\log_4 3} + 4n$

$1 + 2 + \cdots + n = \frac{n(n+1)}{2}$

$= \frac{n^2+n}{2}$ O(

$= n^{.7} + 4n$

$1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$

$= \frac{2n^3 + 2n^2 + }{6}$

$= O(n)$

$O(n^3)$

3. $T(n) = T(n-1) + n^4$

$T(n) = T(n-2) + (n-1)^4 + n^4$

$= T(n-3) + (n-2)^4 + (n-1)^4 + n^4$

$= T(n-4) + (n-3)^4 + (n-2)^4 + (n-1)^4 + n^4$

$= T(n-k) + (n-(k-1))^4 + (n-(k-2))^4 + \cdots + (n-2)^4 + (n-1)^4 + n^4$

$k = n$

$= T(0) + 1^4 + 2^4 + \cdots + (n-2)^4 + (n-1)^4 + n^4$

$= O(n^5)$

4. $T(n) = T(n-1) + n$

$\Rightarrow T(n) = T(n-1) + n$

$T(n) = T(n-2) + (n-1) + n$

$T(n) = T(n-3) + (n-2) + (n-1) + n$

$T(n) = T(n-k) + (n-(k-1)) + \cdots + (n-2) + (n-1) + n$

$n - k - 1 = -1$

$n - k = 0$

$n = k$

$n - k = -1 \Rightarrow n = k$

$T(n) = T(0) + 1 + \cdots + (n-2) + (n-1) + n$

$= 1 + 2 + \cdots + (n-2) + (n-1) + n$

$= \frac{n(n+1)}{2} = \frac{n^2+n}{2} = O(n^2)$

5. $T(n) = 1 + T(n-1)$

$\Rightarrow T(n) = 1 + T(n-1)$

$= 2 + T(n-2)$

$= 3 + T(n-3)$

$= k + T(n-k)$

$n - k = 1 \rightarrow k = n-1$

$T(n) = n - 1 + T(1)$

$= O(n) + O(1)$

$= O(n)$

6. $T(n) = 4T(n/2) + n^2$ $\quad T(1) = 1$

$\Rightarrow T(n) = n^2 + 4T(n/2)$

$= n^2 + 4\left((n/2)^2 + 4T(n/4)\right) = 2n^2 + 4^2 T(n/2^2)$

$= 2n^2 + 4\left((n/4)^2 + 4T(n/2^3)\right) = 3n^2 + 4^3 T(n/2^3)$

$= \cdots kn^2 + 4^k T(n/2^k)$

$2^k = n$

$4^k = n^2$

$n/2^k = 1$

$n = 2^k \Rightarrow \log n = k$

$T(n) = n^2 \log n + n^2 T(0)$

$= n^2 \log n + n^2 = O(n^2 \log n)$

$T(n) = 2T(n/2) + 2n + 3$

$= 3 + 2n + 2\left[3 + 2(n/2) + 2T(n/4)\right]$

$= 3 + 2n + 2\left[3 + n + 2T(n/4)\right] = 9 + 4n + 4T(n/4)$

$= 3^2 + 2^2 n + 2^2\left[3 + 2(n/4) + 2T(n/2^3)\right]$

$= 3^2 + 2^2 n + 12 + 2n + 2^3 T(n/2^3)$

$= 3^3 + 2^3 n + 2^3 T(n/2^3)$

$T(n) = 2^k + 2^k n + 2^k T(n/2^k)$

---

7. $T(n) = \begin{cases} 2 & n = 1 \\ 2T(n/2) + 2n + 3 & n > 1 \end{cases}$

$\Rightarrow T(n) = 3 + 2n + 2T(n/2)$

$= 3 + 2n + 2\left[3 + 2(n/2) + 2T(n/2^2)\right]$

$= 3 + 2\times3 + 2\times2n + 2^2 T(n/2^2)$

$= 3 + 2\times3 + 2\times2n + 2^2 + \left[3 + 2(n/2^2) + 2T(n/2^3)\right]$

$= 3 + 2\times3 + 2^2\times3\times2n + 2^3 T(n/2^3)$

$= \left[3 + 2\times3 + 2^2\times3 + \cdots + 2^{k-1}\times3\right] + \left[k\times2n\right] + 2^k T(n/2^k)$

$n/2^k = 1 \Rightarrow n = 2^k \Rightarrow \log n = k$

$T(n) = 3\left[2^0 + 2^1 + 2^2 + \cdots 2^{k-1}\right] + 2n\log n + n\cdot T(1)$

$= 3\left[(2^k - 1)/(2-1)\right] + 2n\log n + n\cdot 2$

$= 3(n-1) + 2n\log n + 2n = 3n - 3 + 2n\log n + 2n$

$= 5n - 3 + 2n\log n$

$= O(n\log n)$

- Sum of infinite terms of GP $= \frac{a}{1-r}$
- Sum of 1^{st} n terms $= \frac{a(r^n - 1)}{r-1}$

8. $T(n) = \begin{cases} 1 & n=1 \\ c + T(n/2) & n>1 \end{cases}$ $\rightarrow O(\log_2 n)$

9. $T(n) = 2T(n/2) + n^2$ $\rightarrow O(n^2)$

10. $T(n) = \begin{cases} T(n/2) + 1 & n>1 \\ 1 & n=1 \end{cases}$ $\rightarrow \log_2(n) + 1 = O(\log n)$

11. $T(n) = \begin{cases} 1 & \\ 2T(n-1) + 1 & \end{cases}$ $\rightarrow 2^n - 1 = O(2^n)$

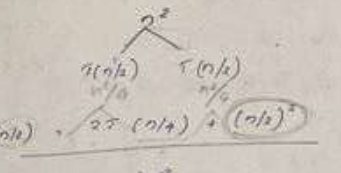
12. $T(n) = T(n/3) + n$ $(3/2)n + 1 = O(n)$

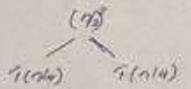
2. Recursion Tree Method :—

- Pictorial representation of an iteration method.
- Which is in the form of a tree where at each level nodes are expanded
- Each node represents the cost of a single sub problem
- We can sum the cost within each level of the tree to obtain a per level cost and then we sum all per level cost to determine the total cost of all levels of the recursion.
- It is more useful when divide & conquer algorithm is used.

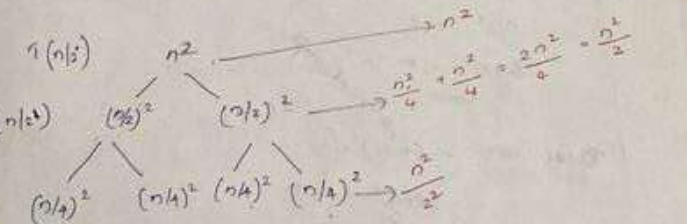
1) $T(n) = 2T(n/2) + n^2$

n^2 is the total cost (divide and combine)





$T(n/4) = 2T(n/8) + (n/4)^2$



$T\left(n/2^k\right)^{2^k} \rightarrow n/2^k$ ground level $T(1)$ $\rightarrow \frac{n^2}{2^k}$.

$n/2^k = 1 \Rightarrow k \Rightarrow \log n$

$$\text{Cost} = \frac{n^2}{2\log_2^n} \cdot \frac{n^2}{n\log_2^2} = \frac{n}{}$$

Sum/cost

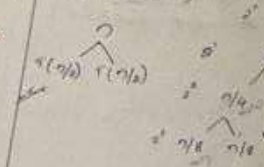
$$T(n) = n^2 + \frac{n^2}{2} + \frac{n^2}{4} + \frac{n^2}{8} \cdots + n^2/2^k$$

$$= n^2\left[1 + (1/2) + (1/2^2) + \cdots + (1/2^k)\right] \text{ no}$$

$$= n^2\left[\left[1 - (1/2)^{k+1}\right] / \left[1 - (1/2)\right]\right]$$

$$= \Theta(n^2)$$

2. $T(n) = 2T(n/2) + n$



Level	Node	Cost
0	$2^0 = 1$	$1 \cdot n$
1	$2^1 = 2$	$2 \cdot n/2$
2	$2^2 = 4$	$4 \cdot n/4$
3	$2^3 = 8$	$8 \cdot n/8$
⋮	⋮	⋮
k	2^k	n
$(k+1)$		

$2^k \; T(1) \quad T(1) \cdots$

$T(n/2^k) = T(1)$

$n/2^k = 1$

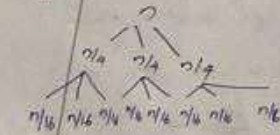
$\log n = k$

Total cost $= (k+1)\, n$

$= (\log n + 1)\, n$

$= n \log n + n = O(n \log n)$

3. $T(n) = 3T(n/4) + n$



$T(n/4) = 3T(n/4) + n/4$

n	1
$3n/4$	
$9n/16$	

$T(1)$ Total cost $= n + \frac{3n}{4} + \frac{9n}{16} + \cdots$

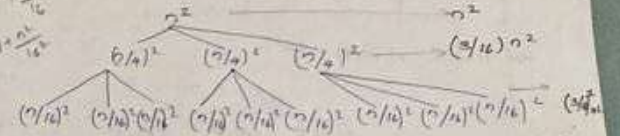
$= n\left(1 + 3/4 + (3/4)^2 + \cdots\right)$

$= n\left(\frac{1}{1-3/4}\right) = 4n = O(n)$

4. $T(n) = 3T(n/4) + n^2$

$3T(n/4^2) + \frac{n^2}{16}$

$3T(n/4^3) + \frac{n^2}{16^2}$



$T(n/4^k) \quad T(1)$ $\qquad (3/16)^k n^2$

$n/4^k = 1 \Rightarrow k = \log_4 n$

$T(n) = n^2 + (3/16)\, n^2 + (3/16)^2 n^2 + \cdots + (3/16)^k n^2$

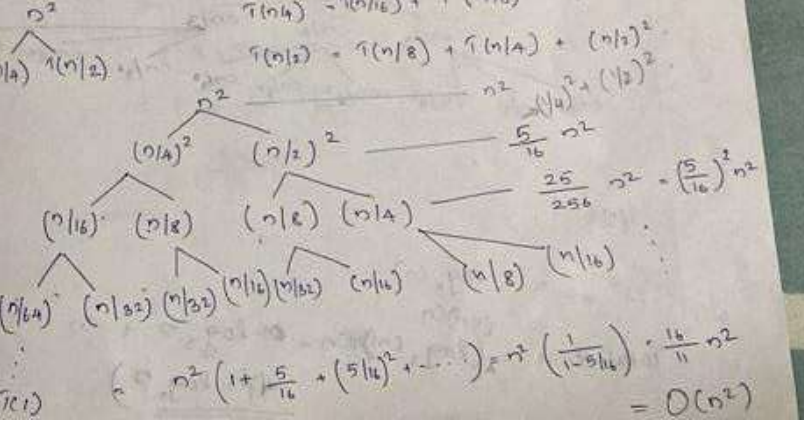
$= n^2 \left[1 + (3/16) + (3/16)^2 + \cdots (3/16)^k\right]$

$= n^2 \left[1/(1 - 3/16)\right] \cdot \; {}^{16}/_{13}\, n^2 = O(n^2)$

5. $T(n) = T(n/4) + T(n/2) + n^2$

$T(n/4) = T(n/16) + T(n/8) + (n/4)^2$

$T(n/2) = T(n/8) + T(n/4) + (n/2)^2$

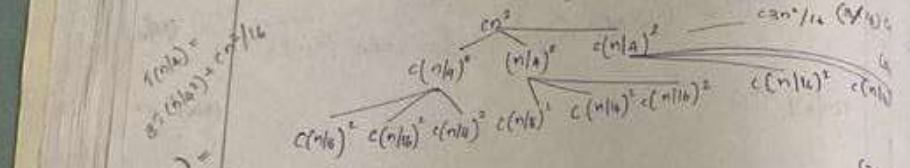


$T(1)$ $\quad = n^2\left(1 + \frac{5}{16} + (5/16)^2 + \cdots\right) = n^2\left(\frac{1}{1 - 5/16}\right) \cdot \frac{16}{11} n^2$

$= O(n^2)$

6. $T(n) = \begin{cases} 1 \\ 3T(n/4) + cn^2 \end{cases}$

$\Rightarrow T(n) = 3T(n/4) + cn^2$



$T(n/4) = 3T(n/16) + 3cn^2/16$

$T(n/4^2) = 3T(n/4^2) + 3cn^2/16^2$

$T(n/4^k)$

$T(1)$

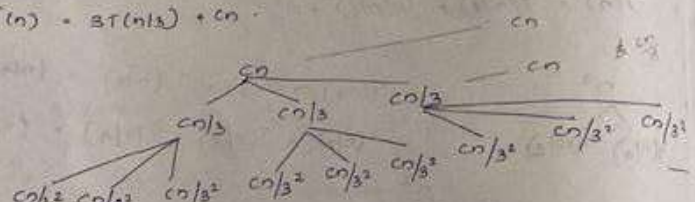
$n/4^k = 1 \quad 4^k = n \quad k = \log_4 n$

$T(n) = cn^2 + (3/16)cn^2 + (3/16)^2 cn^2 + \cdots + (3/16)^k cn^2$

$= cn^2 \left[1 + (3/16) + (3/16)^2 + \cdots + (3/16)^k\right]$ GP

$= cn^2 \left[\frac{1}{1 - (3/16)}\right] = \frac{16}{13} cn^2 = O(n^2)$

7. $T(n) = 3T(n/3) + cn$



$T(n/3) = 3T(n/3^2) + (cn/3)$

$T(n/3^2) = 3T(n/3^3) + (cn/3^2)$

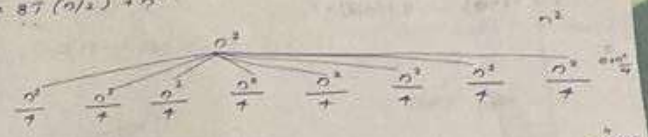
$T(n/3^k)$ . . .

$n/3^k = 1 \Rightarrow k = \log_3(n)$

Total cost $= \sum_{x=1}^{(x+1)cn} (\log_3(n)+1)cn = cn\log_3 n + cn$

$= O(n \log_3 n)$

8. $T(n) = 8T(n/2) + n^2$



$T(1)$

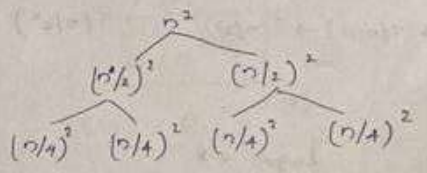
Total cost $= n^2 + 2n^2 + 4n^2 + \cdots + 2^k n^2$

$= n^2 \left[2^0 + 2^1 + \cdots + 2^k\right]$

$= n^2 \left[1 \left(\frac{2^{k+1} - 1}{1}\right)\right] = n^2(2n - 1)$

$= O(n^3)$

9. $T(n) = 2T(n/2) + n^2$



n^2

$2(n/2)^2$

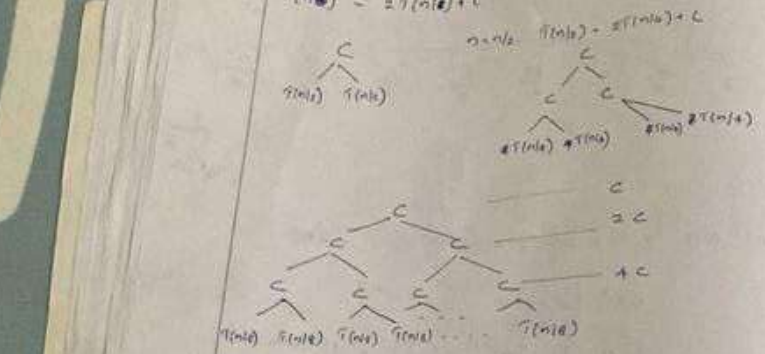
$+ (n/2)^2$

cost $= n^2 + 2(n/2)^2 + \cdots$

$= n^2 \left(1 + 1/2 + 1/4 + \cdots\right)$

$= O(n^2)$

10. $T(n) = 2T(n/2) + c$



$T(n)$ ...

$T(n/2) \to T(n/4) \to T(n/8) \cdots$ k^{th} term

$T(n/2^1) \to T(n/2^2) \to T(n/2^3) \cdots T(n/2^k)$

$n/2^k = 1$

$n = 2^k \qquad \log_2 n = k$

total cost $= c + 2c + 4c + \cdots 2^k c$

$= c(1 + 2 + 4 + \cdots 2^k)$

$= c\left(\frac{2^{k+1} - 1}{2 - 1}\right)$

$= c\left[\frac{2^{\log n} - 1}{1}\right] = c\left[n^{\log_2^2} - 1\right]$

$= c(n-1) = cn - c = O(n)$

Sum of GP: $S_n = \frac{a(r^n - 1)}{r - 1}$

③ master's theorem:—

- The master method provides a 'cook book' method for solving the recurrence of the form

$T(n) = aT(n/b) + f(n)$ $\quad 2T(n/2) + n^2$

1
c

* Theorem:

Let $a \geq 1$ & $b > 1$ be constants, let $f(n)$ be a function and let $T(n)$ be defined on the non-negative integers by the recurrence:

$T(n) = aT(n/b) + f(n)$ where we interpret n/b to mean $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. then $T(n)$ has the following asymptotic bounds

① $f(n) = O(n^{\log_b^a - \varepsilon})$ for some constant $\varepsilon > 0$

use $f(n) \leq n^{\log_b^a}$ $\quad g(n)$ greater

$T(n) = \Theta(n^{\log_b^a})$

② $f(n) = \Theta(n^{\log_b^a})$ then $\quad f(n) = g(n)$

$T(n) = \Theta(n^{\log_b^a} \lg^k n)$

③ $f(n) = \Omega(n^{\log_b^a + \varepsilon})$ $\quad \varepsilon > 0$ if $f(n/b) \leq f(n)$

$g(n)$ is smaller

$T(n) = \Theta f(n)$

? solve the recurrence $T(n) = 9T(n/3) + n$

$T(n) = 9T(n/3) + n \qquad a = 9 \quad b = 3 \quad f(n) = n$

$g(n) = n^{\log_b^a} = n^{\log_3^9} = n^2$

$f(n) < g(n)$

$\therefore T(n) = \Theta(n^{\log_b^a}) = \Theta(n^2)$

2. $T(n) = T(\sqrt{n}/2) + 1$

→ $T(n) = T(n/(2)^2) + 1$ $a = 1$ $b = 2$ $f(n) = 1$

$n^{\log_b^a} = n^{\log_{(2)^2}^1} = n^0 = 1$

$f(n) = n^{\log_b^a} = 1$

case 2

$f(n) = \Theta(n^{\log_b^a})$ then

$T(n) = \Theta(n^{\log_b^a} \log_2 n)$

$\therefore T(n) = \Theta(n^0 \log n)$

$= \Theta(1 \log n) = \Theta(\log n)$

3. $T(n) = 3T(n/4) + n \log_2 n$

$a = 3$ $b = 4$ $f(n) = n \log_2 n$

$n^{\log_b^a} = n^{\log_4^3} = n^{0.793}$

$n \log n > n^{0.793}$

case 3 $f(n) = \Omega(n^{\log_b^a + \varepsilon})$

$T(n) = \Theta(f(n))$

$\therefore T(n) = \Theta(n \log_2 n)$

4. $T(n) = 9T(n/3) + n^2$

$a = 9$ $b = 3$ $f(n) = n^2$

$n^{\log_3^9} = n^2$

$f(n) = n^{\log_3^9}$

case 1

$f(n) = \Theta(n^{\log_b^a})$ → $T(n) = \Theta(n^{\log_b^a} \log n)$

$\therefore T(n) = \Theta(n^2 \log n)$

5. $T(n) = 9T(n/3) + n^3$ $n^{\log_b^a} = n^{\log_3^9} = n^2$

$a = 9$ $b = 3$ $f(n) = n^3$

$f(n) > n^2$

$f(n) = \Omega(n^{\log_b^a + \varepsilon})$ → $T(n) = \Theta(f(n))$

$\therefore T(n) = \Theta(n^3)$

6. $T(n) = 2T(n/2) + (n/\log n)$ → $T(n) = \Theta(n \log(\log n))$

7. $T(n) = 0.5\,T(n/2) + n$

Master's theorem fails because $a \neq 1$ $a = 0.5$

8. $T(n) = 2^n T(n/2) + n^n$

$a \neq 1$ here $a = 2^n$ and is not a constant so we can't solve this relation using master's theorem

9. $T(n) = 5T(n/3) - n \log n$

$f(n)$ is negative so master's theorem doesn't applicable.